

Von: Helen Toner from Rising Tide helentoner@substack.com
Betreff: 2 Big Questions for AI Progress in 2025-2026
Datum: 23. April 2025 um 14:07
An: dr.johannes.meier@gmail.com



Forwarded this email? [Subscribe here](#) for more

2 Big Questions for AI Progress in 2025-2026

On how good AI might—or might not—get at tasks beyond math & coding

HELEN TONER

APR 23



READ IN APP ↗

Pledge your support

One thing I want to do sometimes on Rising Tide is take ideas that are bubbling up inside the AI world and bring them to a broader audience.

Today's post is about one such topic. In technical language, it's about verifiable rewards, generalization, and scaling reasoning training.

For a less technical audience, the way I'd put it is that **how far AI progresses over the next couple of years will be determined in significant part by the answers to these two questions:**

1. Beyond math and coding, where else can you automatically grade answers to hard problems?
2. How much will improving performance in auto-graded areas spill over into strong performance on other tasks?

Auto-verified math and coding problems have been central to recent progress in "reasoning models," and reasoning models are central to predictions that AI will advance rapidly in the next year or two. So the answers to these two

AI will advance rapidly in the next year or two. So the answers to these two questions will go a long way in determining what kind of AI progress we're about to see.

What are reasoning models again?

In September 2024, OpenAI introduced a family of models called o1, which it described as “a new series of reasoning models for solving hard problems.” The term “reasoning models” stuck, and is now used for a range of comparable AI models, including Gemini Flash Thinking from Google and r1 from DeepSeek.¹

To create a reasoning model, you start with a regular language model that has been pre-trained to predict the next token (probably with a dash of post-training to make its answers more helpful and harmless). **Next-token prediction is considered a type of “imitation learning,”** since the optimization pressure of the training setup pushes the models towards recreating patterns in the (largely) human-written text they are trained on.

What distinguishes reasoning models from other large language models is what happens next: a new kind of post-training using reinforcement learning (RL). The setup here is quite different from imitation learning.² Rather than trying to predict what comes next in some sample text, the model is given a problem to solve, and tries to generate a series of tokens (a “chain of thought”) that leads to a solution. When the model manages to produce a chain of thought that leads to a correct answer, it's updated to be more likely to produce similar chains of thought in the future. When it produces an incorrect solution, it's updated to make that kind of generation less likely.

Conceptually, this is quite simple. You just give the model a problem, let it try to find a solution, boost successful attempts and suppress unsuccessful attempts, then repeat until it's very good at solving problems. And it's not a new idea: this kind of RL was behind some of the biggest AI success stories of the 2010s, when AI reached human-level performance and beyond in Atari games, Go, Dota 2, and other games.

Even the idea of using RL to make language models better is not new.³ The challenge is actually getting it to work—and that’s where math and coding come in.

What makes reasoning models so good at math and code

The basic reason why these models are so good at math and coding is... they were trained on a ton of math and coding. But why is *that*?

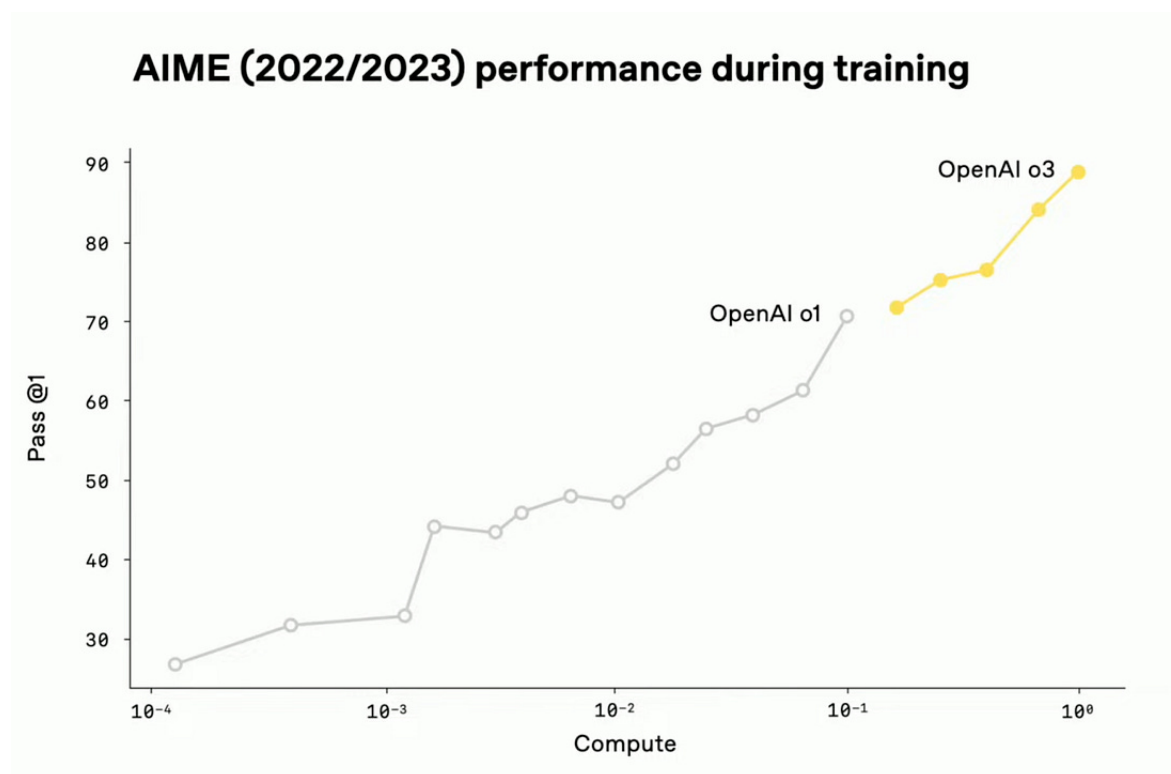
There are multiple reasons. For one thing, these are areas where step-by-step thinking is often helpful; for another, many people see math and coding as very difficult, so it’s especially impressive to do well there. Not to mention that coding has already proven to be a very lucrative use case for language models—and one where the companies have abundant in-house expertise.

But the primary reason why reasoning models are trained so heavily on math, coding, and closely related areas is that these domains let you grade solutions at scale. In math, you can run calculations to check the numbers or use formalized proof checkers to validate the correctness of a proof. In coding, you can run tests to check if a generated program works as intended, or look directly at performance numbers (e.g. speed) to see if they’ve improved.

In the reinforcement learning setup described above, **this kind of automated checking is enormously helpful because it allows you to run the training process at much larger scale.** If you need a human to check every attempt the model makes at solving a problem and determine how well it did, the model will only get as many attempts as you can pay human graders for. This gets expensive quickly—not just in terms of money, which AI companies have plenty of, but also in terms of time. And the more complicated the task, the more specialized the human graders need to be, so mass data annotation by low-skill workers in low-wage countries is not an option.

Automating the process of grading answers and/or generating

Automating the process of grading answers and/or generating problems means that the main determinant of how long you can train your system is computing power. This is a huge part of why questions whose answers can automatically be verified are so appealing to AI developers. With the new supercomputing clusters the leading companies have coming online this year, being able to just dial up the compute and watch the performance numbers tick upwards is a situation they love to be in.



A plot from the [launch livestream](#) for OpenAI's latest reasoning models, o3 and o4-mini, showing increased training compute leading to improved performance on the AIME math benchmark. The voiceover from Wenda Zhou (at 19:05) makes clear that the additional training compute is being used for RL: "As we put in more RL compute, we're also able to get commensurate gains."

Another useful feature of auto-grading is that it makes "inference scaling" potentially far more powerful. Inference scaling refers to the ability to dial up how much compute an already-trained model uses to answer a given question. You can use extra inference compute in two main ways: either to have the model "think" for longer (i.e. generate a longer chain of thought) before reaching an answer, or to have it re-attempt the same problem multiple times and then choose the answer it thinks is best. The latter is most

relevant here—if **you can auto-grade possible answers, you can just let the AI model attempt the problem multiple times and then choose the best answer**, which can be a very powerful technique. This is essentially the approach that let OpenAI score an impressive 88% on the famously difficult ARC-AGI benchmark: they let their o3 model make 1024 attempts at each problem, and then used some (undisclosed) method to choose which answer to submit.

Note how focusing on the importance of auto-verification differs from a claim I heard a lot when the first reasoning models were released, which is that they work well for questions where the answer is clearly correct or incorrect.⁴

“Easy to verify” is not the same as “has a definite answer.” A chemistry question about whether you can synthesize a certain substance using a certain reaction pathway has a definite answer, but it's hard to verify—you need to run real world experiments to find out if it works or not. By contrast, if you ask the model to generate a poem fitting specific constraints (e.g. the Lem test), there's no single correct answer, but it's pretty easy to check if the proposed poem fits the constraints or not. Generating poems that fit constraints is a pretty niche use case, so AI developers don't seem to be focusing on it, but that kind of task would be much more amenable to reasoning training than designing chemical reactions.⁵

Pledge your support

So what are the answers?

Let's return to the two big questions from the top (call them Q1 and Q2):

1. Beyond math and coding, where else can you automatically grade answers to hard problems?
2. How much will improving performance in auto-graded areas spill over into strong performance on other tasks?

Hopefully the relevance of these questions is now clear. Q1 is about which other areas AI developers will manage to make amenable to large-scale

Other areas AI developers will manage to make amenable to large scale reasoning training. Q2 is about how broadly performance might generalize, even if models are only directly trained to reason about a relatively small range of areas. Or to put it another way: Q1 is about what you *can* do at massive scale, and Q2 is about what you *don't need to do* at massive scale.

What speculative answers can we give so far?

Q1: Expanding domains

I don't know how far they've gotten, but it's a safe bet that the top AI companies are doing everything they can to find ways to train at scale on as many domains as they can.

One major strategy is using AI itself to evaluate responses (referred to in technical circles as LLM-as-a-judge). Rather than using a structured validation process (like unit tests for code or a formal proof checker for a mathematical proof), you can simply ask an LLM—perhaps even a different instance of the same LLM that you're training—to evaluate a proposed solution. The big advantage of this approach is its flexibility: you can ask an LLM to evaluate just about anything. The big disadvantage is that there's no guarantee the LLM's evaluation will track what you care about, whether that's objective correctness (e.g. whether a proposed chemical synthesis pathway will work) or something squishier like expert judgment (e.g. the quality of a legal analysis). **It's essentially just an empirical question whether using an LLM as your evaluation method will happen to work for any given type of problem**, and you can bet the big AI developers are throwing all kinds of LLM-judged spaghetti at the wall to see where this works.

(If it seems surprising that an LLM could assess performance on a task it can't perform well itself, remember that humans do this all the time. I'm a terrible designer, but I can distinguish a high-quality logo design from an amateur one. Likewise, managers frequently have to evaluate work by their employees that they could not have done themselves. In AI, this gets referred to as **the “generation-verification gap,”**⁶ which is the gap between how hard it is to create a piece of work and how hard it is to determine whether the work is good.)

Another type of training the companies are trying to make work is for so-called AI agents: using RL and chain-of-thought reasoning to train LLM-based systems to carry out useful tasks. There's much more to be said about LLM-based agents, but for the purposes of this post, the relevant question is: Can you create a set of tasks for an agent to try where you can automatically check if they succeeded? If agents were in widespread use, this could actually be quite easy. Each time a user's agent tried something, you could just give the user the option to hit a checkmark (success!) or a cross (didn't work), then train the next-generation agent to do better. But to get that kind of data at scale, you need to start with an agent that's useful enough for people to use it a lot—a threshold today's agents do not seem to have crossed.

In the absence of real-world agent use, companies are likely creating hand-curated sets of tasks where a human (or perhaps LLM-as-a-judge) grades the agent's success. They're also looking for tasks that lend themselves to automated grading—e.g. in the recent release of o3 and o4-mini, OpenAI described training both models to reason about *which tools* to use for a given task, perhaps using a hand-curated dataset of which tools are appropriate for which task, or perhaps using some auto-graded approach. This kind of experimentation is sure to continue.

Q2: Generalization

We're clearly seeing at least some spillover ("generalization"), where training on math and coding makes models better in other areas, though it's not clear how far it will go. Even before the advent of reasoning models, researchers found that adding more code data at the pre-training phase led to surprising improvements on a range of non-code problems requiring logical thinking.⁷ As for reasoning models, **the ability to (sometimes) notice when they have made a mistake, backtrack, and retry the problem is a very general tactic that they seem to have learned in their math- and code-heavy training.** The ability to break a problem down into smaller pieces, solve each in turn, and combine the pieces into a coherent answer to the original question is another example of something that should be very broadly applicable

approach.

There are some other interesting signs of generalization, though it's hard to be sure because so little information is usually released about how new products are trained. OpenAI's Deep Research, for example, seems to be an example of reasoning generalizing beyond math and code—it specializes in web research, not problem-solving per se, and has received some strong reviews. OpenAI says “Deep research was trained on new browsing datasets created specifically for research use cases. The model learned [...] through reinforcement learning training on these browsing tasks.” But does this mean that they found ways to do large-scale auto-verified training on these tasks, which would count as progress on Q1 (expanding domains), not Q2 (generalization)? Or did they find that they could do much smaller scale training on Deep Research's specific domain, because o3 (the underlying model)'s math- and code-heavy training made the model much more able to learn to do well in new areas? I'm not sure.

It's worth noting how little AI researchers understand about what's going on with generalization more broadly. AI models (especially LLMs) often turn out to be useful on tasks beyond what they were directly trained for, but we don't really have a principled understanding of why this is, nor can we predict how well a given model's performance on one task is likely to generalize to another task. Q2 from this post is just one of many interesting open questions about generalization in AI.

For AI to keep advancing rapidly over the next couple of years, at least one of these two big questions has to go quite well for AI developers. If Q1 goes the way developers hope, they'll have a bunch of new areas where they can scale up their RL training, and users will be able to use high-compute inference setups to generate tons of possible answers then automatically choose the best one. If they get lucky on Q2, they'll be able to just stick with scaling the areas they've already figured out, and strong performance in a wide range of other areas will come along for free.

We're clearly *not* in the world where both Q1 and Q2 are a total bust—

and I think that's a significant factor in why so many AI researchers are predicting we're in for an impressive year or two. As described above, developers have already found ways to set up auto-gradable questions in some new areas, and we're seeing some notable spillover into other areas as well. But it remains to be seen just how far either of these trends will go.

I find it interesting how different people's intuitions on Q1 and Q2 seem to be. In machine learning circles, many people expect that some combination of a) scaling up math and code training, b) managing to auto-verify other domains so you can scale them up too, and c) reaping the rewards of generalization elsewhere might be all you need to create AI that is extremely intelligent across the board.⁸ When I talk to less technical people they tend to be far more skeptical, often in part because they are unfamiliar with existing evidence about how far auto-grading can get and what kind of generalization we already see.

Personally, my best guess is that (Q1) we find new ways to set up auto-verified learning for some additional domains of interest, though overall a small proportion of what we want AI to be able to do; then (Q2) this results in enough generalization that you only need to train on smaller, hand-curated datasets to get excellent performance on a much wider range of domains.⁹ But even within this prediction, there's lots of room between a version where you get great generalization and only need relatively few hand-curated examples on relatively broadly-scoped topics (e.g. "be a personal assistant," "manage an online business,"), vs. a version where generalization is more limited and the hand-curated training has to be much more fine-grained (e.g. separate training for "book a flight," "schedule a meeting," "brief me on my day," etc.). I don't know what I think is most likely within this range of possibilities.

There's one last reason why I'll be watching domains where answers can be auto-graded: these are **often the domains where it's most feasible for AI to go far beyond human performance.** AlphaZero's self-play setup is what let it wipe the floor with the best human Go players—since it could keep learning to beat itself, human-level performance was an irrelevant threshold. The next-token prediction paradigm used in pre-training naturally converges

towards recreating human-level performance. For better or for worse, reinforcement learning pointed towards auto-verified solutions could reach far higher heights.

Sign up to receive new posts from Rising Tide:

Pledge your support

-
- 1 We're likely headed towards a world where reasoning models and regular LLMs are not cleanly separated from each other; Anthropic's Claude 3.7, which can vary how much "reasoning" it does depending on the question, is a preview of this. But for now the term "reasoning model" is in widespread use.
 - 2 To be fully accurate, it appears that it can be helpful for reasoning models to start with some imitation learning before moving to RL. The DeepSeek-R1 technical report describes training working best when they first fine-tuned the model to imitate thousands of examples of long chains of thought ("cold-start" data) before starting RL. Whether this will always be the best approach, or even whether it's the approach used by companies who don't publish their methods (e.g. OpenAI, Google, Anthropic) is not clear.
 - 3 E.g. a quick search turned up this May 2023 post from Yuxi Li. There are probably earlier public writings, and in private conversations AI researchers were talking informally about doing "AlphaGo for LLMs" pretty much as soon as LLMs were a thing.
 - 4 See e.g. Dario Amodei saying reasoning models work best on "objectively measurable" tasks, or Kevin Roose saying they do well on tasks with a "definite right answer."
 - 5 And in fact the Allen Institute for AI (which shares more about its model development processes than most organizations building reasoning models) has shared data they used for reinforcement learning on their

Nov 2024 Tulu 3 release, which included "verifiable instruction-following tasks." These are tasks that are specifically constructed so that they have automatically verifiable constraints, such as asking the model to generate answers of a certain length or that do/don't include certain keywords.

- 6 Alternatively the "generation-discrimination gap" or the "generation-evaluation gap."
- 7 E.g. from Liang et al. 2022: *"For reasoning-intensive scenarios, we find that the code models, especially code-davinci002, consistently outperform the text models, even on synthetic reasoning scenarios posed in natural language."*

See also Ma et al. 2023.

- 8 Examples of this view include:
 - Lambert 2025: *"A realistic outcome for reasoning heavy models in the next 0-3 years is a world where:*
 - *Reasoning trained models are superhuman on tasks with verifiable domains, like those with initial progress: Code, math, etc.*
 - *Reasoning trained models are well better in peak performance than existing autoregressive models in many domains we would not expect and are not necessarily verifiable.*
 - *Reasoning trained models are still better in performance at the long-tail of tasks, but worse in cost given the high inference costs of long-context."*
 - Kokotajlo et al. 2025: *"Early versions of [iterated distillation and amplification] have been working for many years on easily verifiable tasks, like math and coding problems that have a clear answer, because the techniques used to amplify models often rely on access to some ground truth signal of accuracy. [In our forecast], the models have become sufficiently good at verifying more subjective things (e.g. the quality of a work product), allowing the use of IDA to*

improve the model at many tasks.”

- Szegedy 2019: “In this paper, it is argued that autoformalization is a promising path for systems to learn sophisticated, general purpose reasoning in all domains of mathematics and computer science.”

9 In other words, language models will continue to be few-shot learners, though “few shot” in my guess could mean hundreds or thousands of examples per hand-curated domain, not just a couple examples in the context window.

Thanks for reading. I’d always love to hear your thoughts—feel free to hit reply.

Share

♡ LIKE

💬 COMMENT

↺ RESTACK

© 2025 Helen Toner

548 Market Street PMB 72296, San Francisco, CA 94104

[Unsubscribe](#)

🚩 Start writing